Zonexchoose Project Documentation

1. Project Synopsis

Project Name

Zonexchoose

Objective

Zonexchoose is a full-stack digital voting platform that manages voter onboarding, candidate verification, ballot casting, turnout tracking, and controlled result publication.

Problem It Solves

From the codebase, this project solves a specific election workflow problem:

- only pre-approved voters should be allowed to register
- voters should verify identity before account creation
- candidates should be invited and verified before appearing on the ballot
- each voter should cast only one vote
- turnout should be visible without exposing results early
- administrators should control the election lifecycle from a single interface

Key Features Implemented

- pre-approved voter registry
- OTP-based voter registration
- JWT-based login for voters and admins
- role-based access control
- admin dashboard for election controls
- candidate invitation by email
- token-based candidate document upload
- unique candidate ID verification during upload
- one-vote-per-user enforcement
- turnout analytics
- backend-controlled result visibility
- vote confirmation email after successful submission
- accessibility-focused UI with voice and keyboard support

Target Users

- administrators who manage election setup and governance
- approved voters who register, log in, and vote
- invited candidates who upload verification documents

Real-World Use Case

Based on the workflows in the code, this project fits a controlled institutional election such as:

- college student elections
- department representative elections
- club or campus body voting

- small organizational decision platforms with verified participants

2. Project Overview Based On Code

End-To-End Flow

The system uses a server-rendered static frontend with a JSON API backend.

1. Express serves the HTML, CSS, JS, and uploaded assets.
2. Frontend pages call backend APIs through a shared `App.api()` helper.
3. Authentication is handled with JWT Bearer tokens stored in browser localStorage.
4. MongoDB stores users, approved voter records, candidates, votes, and election configuration.
5. Admin users manage the election from `admin.html`.
6. Voters register through OTP, log in, and vote from `dashboard.html` and `vote.html`.
7. Candidates use tokenized upload links to submit verification documents.

Frontend Flow

- `index.html` is the public landing page.
- `register.html` drives a 3-step OTP registration flow.
- `login.html` and `admin-login.html` authenticate different roles.
- `dashboard.html` shows voter status and turnout.
- `vote.html` loads approved candidates and submits one vote.
- `upload-docs.html` allows invited candidates to upload verification files.
- `admin.html` is the admin control panel for voters, candidates, election status, and results.

All API communication is triggered from browser scripts:

- `client/js/main.js` provides shared API, session, theme, voice, and UI helpers.
- `client/js/auth.js` manages login, OTP, registration, dashboard, and candidate upload flows.
- `client/js/admin.js` handles admin panel behavior.
- `client/js/vote.js` handles ballot loading and vote submission.

Backend Flow

The backend follows a standard Express structure:

- `routes` define endpoints
- `controllers` contain request logic
- `models` define MongoDB collections
- `middleware` handles auth, uploads, and errors
- `utils` provide OTP, mail, sanitization, and startup helpers

Example flow for voting:

1. `client/js/vote.js` submits `POST /api/vote/cast`
2. `server/routes/voteRoutes.js` maps the request
3. `server/middleware/auth.js` validates the JWT

4. `server/controllers/voteController.js` verifies election status, role, duplicate vote status, and candidate validity
5. `Vote` and `User` collections are updated
6. response is returned to the frontend

Database Usage

MongoDB is used through Mongoose models:

- `User` stores registered accounts
- `ApprovedVoter` stores pre-approved voter identities and OTP state
- `Candidate` stores invited and reviewed candidate records
- `Vote` stores final ballot submissions
- `VotingConfig` stores election-wide settings such as voting enabled and results visible

Unique indexes help protect data integrity:

- unique email and student ID for `User`
- unique `(email, studentId)` for `ApprovedVoter`
- unique `(email, studentId, position)` for `Candidate`
- unique `voter` in `Vote`

3. Complete Folder Structure

This reflects the actual project structure in the workspace.

```text
zonexselect/
+-- client/
|   +-- assets/
|   |   +-- logo.svg
|   +-- css/
|   |   +-- styles.css
|   +-- js/
|   |   +-- admin.js
|   |   +-- auth.js
|   |   +-- main.js
|   |   +-- vote.js
|   +-- admin-login.html
|   +-- admin.html
|   +-- dashboard.html
|   +-- index.html
|   +-- login.html
|   +-- register.html
|   +-- upload-docs.html
|   +-- vote.html
+-- docs/
|   +-- PROJECT_DOCUMENTATION.md
+-- node_modules/
+-- server/
|   +-- config/
|   |   +-- db.js
|   +-- controllers/
|   |   +-- adminController.js
|   |   +-- authController.js
|   |   +-- candidateController.js
|   |   +-- voteController.js
```

```
|   +-- middleware/
|   |   +-- auth.js
|   |   +-- errorHandler.js
|   |   +-- upload.js
|   +-- models/
|   |   +-- ApprovedVoter.js
|   |   +-- Candidate.js
|   |   +-- User.js
|   |   +-- Vote.js
|   |   +-- VotingConfig.js
|   +-- routes/
|   |   +-- adminRoutes.js
|   |   +-- authRoutes.js
|   |   +-- candidateRoutes.js
|   |   +-- voteRoutes.js
|   +-- uploads/
|   |   +-- 1776922149821-phpApart.pdf
|   |   +-- 1776922150552-lord-shiva-3840x2160-14623.jpg
|   |   +-- 1776922775096-phpApart.pdf
|   |   +-- 1776922775307-car.jpg
|   +-- utils/
|   |   +-- mailer.js
|   |   +-- otp.js
|   |   +-- sanitize.js
|   |   +-- seed.js
|   +-- server.js
+-- .env
+-- .gitignore
+-- package-lock.json
+-- package.json
+-- README.md
```

4. Detailed Folder Explanation

`client/`

Contains the browser-facing application.

- purpose: renders the UI and sends API requests to the backend
- connects to: `server/server.js` for static serving and `/api/*` routes for data
- example files: `admin.html`, `vote.html`, `js/main.js`, `css/styles.css`

`client/assets/`

Contains static design assets.

- purpose: stores reusable media used by pages
- connects to: HTML pages through normal static asset loading
- example files: `logo.svg`

`client/css/`

Contains global application styling.

- purpose: visual design, layout system, responsive behavior, animations, theme variants
- connects to: every HTML page through `/css/styles.css`
- example files: `styles.css`

`client/js/`

Contains all browser-side behavior.

- purpose: API calls, form handling, session management, page logic, shared UI helpers
- connects to: HTML pages, backend API routes, browser localStorage
- example files: `main.js`, `auth.js`, `admin.js`, `vote.js`

`docs/`

Contains project documentation.

- purpose: onboarding, architecture understanding, developer handoff
- connects to: the rest of the repository by describing how each layer works
- example files: `PROJECT_DOCUMENTATION.md`

`node_modules/`

Contains installed dependencies.

- purpose: local package storage for runtime and development libraries
- connects to: both frontend serving and backend runtime through `package.json`
- example packages: `express`, `mongoose`, `jsonwebtoken`, `nodemailer`, `multer`

`server/`

Contains the backend API and server logic.

- purpose: handles routing, business logic, database access, email, uploads, and static file serving
- connects to: frontend pages, MongoDB, and mail provider
- example files: `server.js`, `controllers/*`, `models/*`

`server/config/`

Contains configuration logic.

- purpose: centralized database connection setup
- connects to: `server.js`
- example files: `db.js`

`server/controllers/`

Contains request-handling logic between routes and database.

- purpose: business rules and workflow execution
- connects to: `routes`, `models`, and `utils`
- example files: `authController.js`, `voteController.js`

`server/middleware/`

Contains reusable Express middleware.

- purpose: authentication, upload handling, centralized error formatting
- connects to: routes and controllers
- example files: `auth.js`, `upload.js`, `errorHandler.js`

`server/models/`

Contains Mongoose schemas and collections.

- purpose: define data structure, constraints, and indexes
- connects to: controllers for persistence and queries
- example files: `User.js`, `Vote.js`, `Candidate.js`

`server/routes/`

Contains endpoint definitions.

- purpose: map API URLs to controller methods and middleware
- connects to: `server.js` and `controllers`
- example files: `authRoutes.js`, `adminRoutes.js`

`server/uploads/`

Contains uploaded candidate documents and photos.

- purpose: local disk storage for nomination verification files
- connects to: `upload.js`, `candidateController.js`, and static `/uploads` route
- example files: uploaded PDF and image assets currently present in the project

`server/utils/`

Contains helper utilities.

- purpose: OTP generation, input sanitization, email transport, startup seeding
- connects to: controllers and startup code
- example files: `mailer.js`, `otp.js`, `seed.js`

5. File-Level Explanation

Root Files

`package.json`

- defines project metadata, scripts, and dependencies
- why it exists: it is the runtime manifest for Node.js
- key items:
  - `start` runs `node server/server.js`
  - `dev` runs `nodemon server/server.js`
  - dependencies include Express, Mongoose, JWT, Multer, Nodemailer

`.env`

- stores environment-specific configuration
- why it exists: keeps deployment settings outside code
- used by:
  - `MONGO_URI`
  - `JWT_SECRET`
  - email settings
  - admin bootstrap credentials
  - `CLIENT_URL`

`README.md`

- provides quick-start documentation
- why it exists: first-touch documentation for developers and reviewers

Frontend Files

`client/index.html`

- public landing page
- why it exists: introduces the platform and directs users to register or log in
- connects to:
  - `client/js/main.js`
  - shared CSS
  - auth navigation

`client/login.html`

- voter login page
- why it exists: authenticates voter accounts only
- key behavior:
  - submits to `/api/auth/login`
  - redirects valid voter users to dashboard

`client/admin-login.html`

- administrator login page
- why it exists: separates admin entry point from voter flow
- key behavior:
  - submits to `/api/auth/login`
  - rejects non-admin role on the client

`client/register.html`

- three-step voter onboarding page
- why it exists: implements OTP request, OTP verification, and account creation
- key behavior:
  - step 1 calls `/api/auth/send-otp`
  - step 2 calls `/api/auth/verify-otp`
  - step 3 calls `/api/auth/register`

`client/dashboard.html`

- logged-in voter home page
- why it exists: shows turnout, vote status, and voting availability
- key behavior:
  - fetches current user through auth guard

  - fetches turnout from `/api/vote/turnout`

`client/vote.html`

- ballot page for voters
- why it exists: displays approved candidates and submits a vote
- key behavior:
  - loads approved candidates from `/api/vote/candidates`
  - posts selected candidate to `/api/vote/cast`

`client/upload-docs.html`

- candidate document submission page
- why it exists: supports nomination verification via secure invitation token
- key behavior:
  - validates token with `/api/candidates/by-token/:token`
  - submits multipart upload to `/api/candidates/upload/:token`

`client/admin.html`

- administrator control panel
- why it exists: central workspace for managing the election lifecycle
- key behavior:
  - controls election flags
  - adds approved voters
  - invites candidates
  - approves or rejects nominations
  - loads protected results

`client/js/main.js`

- shared frontend framework for the app
- why it exists: centralizes common browser-side behavior
- key functions:
  - `api()` for fetch-based API communication
  - `requireAuth()` for protected page access
  - `setSession()` and `clearSession()` for localStorage session handling
  - theme and voice toggles
  - toast and loader helpers

`client/js/auth.js`

- handles authentication and user onboarding screens
- why it exists: groups pages related to login, registration, dashboard, and candidate upload
- key functions:
  - `setupLoginPage()`
  - `setupAdminLoginPage()`
  - `setupRegisterPage()`
  - `setupDashboardPage()`
  - `setupUploadDocsPage()`

`client/js/admin.js`

- powers the admin dashboard
- why it exists: handles admin-only workflows on the browser side
- key functions:

  - `setupAdminPage()`
  - `loadSummary()`
  - `renderAdminSummary()`
- connects to:
  - `/api/admin/summary`
  - `/api/admin/approved-voters`
  - `/api/admin/add-candidate`
  - `/api/admin/approve-candidate`
  - `/api/admin/reject-candidate`
  - `/api/admin/toggle-voting`
  - `/api/admin/results`

`client/js/vote.js`

- handles ballot rendering and vote submission
- why it exists: isolates voting-specific UI logic
- key functions:
  - `setupVotePage()`
  - `selectCandidate()`
- behavior:
  - fetches candidates and turnout
  - manages submit state
  - prevents empty submissions
  - locks the UI after successful voting

`client/css/styles.css`

- global visual system for the application
- why it exists: defines consistent UI style and responsive layout rules
- includes:
  - glassmorphism layout
  - cards, forms, buttons
  - responsive grid behavior
  - reduced-motion support
  - theme variables

Backend Files

`server/server.js`

- main Express application entry point
- why it exists: wires together middleware, routes, static serving, startup, and health check
- key responsibilities:
  - loads environment variables
  - applies Helmet, CORS, JSON parsing
  - rate-limits auth routes
  - serves `/api/*` routes
  - serves `client/` and `/uploads`
  - connects to MongoDB and seeds defaults

`server/config/db.js`

- MongoDB connection bootstrap
- why it exists: isolates Mongoose connection logic
- connects to: `server.js`

`server/routes/authRoutes.js`

- maps auth-related endpoints
- routes:
  - `POST /send-otp`
  - `POST /verify-otp`
  - `POST /register`
  - `POST /login`
  - `GET /me`

`server/routes/adminRoutes.js`

- maps admin-only endpoints
- protected by: `protect` and `adminOnly`
- routes cover summary, voter management, candidate management, election toggles, and results

`server/routes/voteRoutes.js`

- maps voting endpoints
- routes:
  - `GET /candidates`
  - `POST /cast`
  - `GET /turnout`

`server/routes/candidateRoutes.js`

- maps public token-based candidate upload endpoints
- routes:
  - `GET /by-token/:token`
  - `POST /upload/:token`

`server/controllers/authController.js`

- handles OTP registration and login lifecycle
- why it exists: contains all auth and identity onboarding logic
- key functions:
  - `sendOtp()`
  - `verifyOtp()`
  - `register()`
  - `login()`
  - `me()`

`server/controllers/adminController.js`

- handles admin workflows
- why it exists: contains all governance, configuration, and review operations
- key functions:
  - `getSummary()`
  - `addApprovedVoter()`
  - `addCandidate()`
  - `approveCandidate()`
  - `rejectCandidate()`
  - `toggleVoting()`
  - `getResults()`

`server/controllers/candidateController.js`

- handles candidate invitation lookup and document upload
- why it exists: separates token-driven candidate operations from admin and auth concerns
- key functions:
  - `getCandidateByToken()`
  - `uploadDocuments()`
- notable logic:
  - validates invitation token expiry
  - requires candidate ID entry
  - checks uploaded government ID filename includes candidate ID
  - removes invalid uploaded files on failed validation

`server/controllers/voteController.js`

- handles vote data and turnout
- why it exists: centralizes election ballot logic
- key functions:
  - `getCandidates()`
  - `castVote()`
  - `getTurnout()`
- notable logic:
  - checks voting enabled
  - checks voter role
  - prevents duplicate voting
  - stores vote and voter snapshot
  - sends confirmation email after successful vote

`server/middleware/auth.js`

- verifies JWT tokens and role access
- why it exists: protects private endpoints
- key exports:
  - `protect`
  - `adminOnly`

`server/middleware/upload.js`

- configures Multer for candidate uploads
- why it exists: handles accepted file types, size limits, and disk storage
- allowed types:
  - PDF
  - JPG
  - PNG

`server/middleware/errorHandler.js`

- standardizes error responses
- why it exists: keeps controller code cleaner and centralizes duplicate and upload error handling

`server/models/User.js`

- registered account schema
- why it exists: stores users who can log in
- key fields:
  - `email`
  - `studentId`

  - `password`
  - `role`
  - `hasVoted`
  - `isActive`

`server/models/ApprovedVoter.js`

- pre-registration voter whitelist
- why it exists: restricts who can register and tracks OTP state
- key fields:
  - `isUsed`
  - `isActive`
  - `otpHash`
  - `otpExpiresAt`
  - `otpVerifiedAt`

`server/models/Candidate.js`

- candidate nomination schema
- why it exists: stores invited candidates, uploaded documents, and review state
- key fields:
  - `status`
  - `invitationToken`
  - `governmentIdPath`
  - `photoPath`
  - `verificationNotes`
  - `rejectionReason`

`server/models/Vote.js`

- ballot submission schema
- why it exists: stores final cast votes
- key design choice:
  - `voter` is unique, enforcing one vote per user at database level

`server/models/VotingConfig.js`

- election-wide configuration schema
- why it exists: stores state that applies to the whole election
- key fields:
  - `votingEnabled`
  - `showResults`
  - `nominationEnabled`

`server/utils/otp.js`

- OTP and invitation token helper
- why it exists: keeps token generation logic reusable
- key functions:
  - `generateOtp()`
  - `hashOtp()`
  - `generateInvitationToken()`

`server/utils/sanitize.js`

- input cleaning and validation helper
- why it exists: normalizes and validates user-provided fields

- key functions:
  - `sanitizeString()`
  - `sanitizeEmail()`
  - `sanitizeStudentId()`
  - `isValidEmail()`
  - `isStrongPassword()`
  - `isValidName()`

`server/utils/mailer.js`

- wraps Nodemailer transport creation and sending
- why it exists: centralizes email delivery for OTP, invitations, approvals, rejections, and vote confirmation
- behavior:
  - uses Gmail transport if credentials exist
  - falls back to stream transport otherwise

`server/utils/seed.js`

- startup seeding helper
- why it exists: ensures a `VotingConfig` exists and optionally bootstraps an admin account

6. Authentication Flow

Login Flow

Voter Login

1. User opens `login.html`.
2. `client/js/auth.js` captures email and password.
3. Frontend calls `POST /api/auth/login`.
4. `authController.login()` validates credentials using bcrypt.
5. Backend returns a JWT and user object.
6. Frontend stores token and user in localStorage.
7. User is redirected to `dashboard.html`.

Admin Login

1. User opens `admin-login.html`.
2. Frontend submits credentials to `POST /api/auth/login`.
3. Backend returns token and role.
4. Frontend rejects non-admin users and redirects them to the voter login page.
5. Valid admin users are redirected to `admin.html`.

Registration Flow

1. Admin creates an approved voter record.
2. Voter submits email and student ID to `POST /api/auth/send-otp`.
3. Backend verifies the record exists and is unused.
4. OTP is generated, hashed, stored on the approved voter record, and emailed.
5. Voter submits OTP to `POST /api/auth/verify-otp`.
6. Backend compares the submitted code to the hashed OTP.
7. Backend returns a short-lived registration token.
8. Voter submits final name, password, and registration token to `POST /api/auth/register`.

9. Backend validates the token, password, and voter record.
10. `User` is created and the approved voter record is marked as used.

JWT Handling

- JWT is created in `authController.js`
- token payload includes `id` and `role`
- expiry is `1d`
- frontend stores token in localStorage as `zonex_token`
- protected API requests send `Authorization: Bearer <token>`
- `auth.js` middleware validates token and loads user from database

Role-Based Access

- `adminOnly` middleware protects admin routes
- frontend also performs role checks in `App.requireAuth()`
- voter pages reject admin accounts
- admin pages reject voter accounts

7. System Design Explanation

Frontend To Backend Connection

The frontend does not use a separate SPA framework. Instead:

- Express serves static HTML files
- each page loads one or more JavaScript files
- shared browser logic uses `fetch()` through `App.api()`
- API base path is `/api`

API Communication

Request pattern:

1. user action happens in the browser
2. JS builds payload
3. `App.api()` sends request
4. backend route matches request
5. controller performs validation and DB work
6. JSON response returns to browser
7. page updates UI or shows error toast

Data Flow Example

Vote submission flow:

1. user selects candidate on `vote.html`
2. `vote.js` sends `POST /api/vote/cast`
3. JWT is attached by `main.js`
4. `voteRoutes.js` forwards request to `castVote`
5. `auth.js` middleware loads the authenticated user
6. `voteController.js` validates voting state and duplicate status
7. `Vote` is created and `User.hasVoted` is updated
8. response message is returned and shown in the UI

Error Handling

- client side:

  - `App.api()` throws errors for non-2xx responses
  - pages show toast messages or empty-state messages
- server side:
  - controllers call `next(error)` for unexpected issues
  - `errorHandler.js` formats Multer errors, duplicate key errors, and generic failures

8. Mobile Responsiveness Check

Current Status

The UI is moderately mobile-friendly.

Evidence from the code:

- responsive Bootstrap grid is used across pages
- `styles.css` includes mobile breakpoints at `991px` and `767px`
- buttons in `.nav-toggle-group`, `.hero-actions`, and `.button-row` become full width on smaller screens
- tables are wrapped with `.table-shell` and `overflow-x: auto`
- layout grids use `auto-fit` and `minmax`
- typography uses `clamp()` for headings

Strengths

- public landing page supports collapsed navigation
- cards and statistics stack well on narrow screens
- forms use large touch-friendly controls
- horizontally scrollable tables avoid layout breakage

Improvements Needed

- other pages do not use a collapsible navbar menu like `index.html`
- long admin pages can become dense on mobile
- admin tables could benefit from card-based mobile layouts
- some control panels place many actions side by side on small screens
- candidate and admin management screens would improve with better spacing and sticky section navigation on mobile

9. Scalability Analysis

Can It Handle 100 Users?

Yes, likely.

Why:

- simple Express + MongoDB architecture is enough for small institutions
- JWT auth is lightweight
- database uniqueness helps protect core integrity
- frontend is static and inexpensive to serve

Can It Handle 1000 Users?

Possibly, with careful hosting, but not comfortably under high concurrency.

Why:

- still a single Node.js process
- no background job queue for email
- vote confirmation emails run inside request handling
- no caching or pagination
- turnout and summary endpoints perform direct counts and full list queries

Can It Handle 10,000 Users?

Not reliably in its current form.

Why:

- no load balancing
- no horizontal scaling strategy in code or deployment
- uploads are stored on local disk
- no Redis, queue, or worker separation
- no CDN or object storage for uploaded assets
- no request throttling on vote and admin-heavy endpoints
- no database transaction around multi-step vote persistence

Current Limitations

- single server instance
- synchronous email in request path
- local file uploads
- no caching layer
- no asynchronous processing
- no observability or metrics layer
- no automated performance testing
- no pagination for admin list endpoints
- summary endpoints return full candidate and voter datasets

Improvements Recommended

- move emails to a background queue
- add Redis for queueing and caching
- use object storage for uploads instead of local disk
- add pagination for admin lists
- add read caching for turnout and summaries
- add database transactions for vote save plus `hasVoted` update
- add horizontal scaling behind a load balancer
- add rate limiting to more than just auth routes
- add monitoring and structured logs
- add load tests before production rollout

10. README File Generation

A refreshed README has been generated in the root of the project:

- `README.md`

It includes:

- project introduction
- feature summary
- setup steps

- tech stack
- folder explanation
- flow summaries
- strengths and limitations

Closing Assessment

This is a well-structured small-to-medium full-stack academic voting application with:

- clear separation of concerns
- working identity and voting flows
- admin governance controls
- useful accessibility features

Its strongest fit is a controlled institutional election rather than a large public voting platform. The architecture is solid for learning, demos, and moderate real-world usage, but it would need performance, reliability, and deployment improvements before large-scale production use.